

# Design Specification

## QSPI Flash Controller with Execute-In-Place (XIP)

### 1. Document Information

| Field                   | Detail                                                                                                                                             |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Project Name            | QSPI Flash Controller with Execute-In-Place (XIP)                                                                                                  |
| Project Engineer        | G Gokul / Gokul G                                                                                                                                  |
| Version                 | v1.0                                                                                                                                               |
| Date                    | 22 September 2026                                                                                                                                  |
| Reference Specification | Macronix MX25U6432FM2I02 QSPI NOR flash, as named in the project document. No datasheet revision is stated in the project document. See Section 8. |

### 2. Overview

The QSPI Flash Controller connects a host to a Macronix MX25U6432FM2I02 QSPI NOR flash. The top-level integration described in the project document is `top_module`. The same document's file list names the integration file `top_module.v`.

There are two ways to talk to the flash.

1. The **indirect/APB path** is a generic command-transfer engine. Firmware programs the command, address, length, and transfer-configuration registers over APB and pulses a start bit. This path is used for WREN, PP, SE, BE, WRSR, RDID, RDSR, and the manual read variants FAST\_READ, QOR, DOR, DIO, and QIOR.
2. The **XIP/AHB path** serves a CPU that fetches code or data over AHB-Lite from a 16-byte prefetch line. On a hit, the line is returned to the AHB master. On a miss, an autonomous engine (`xip_command_engine`) issues an RDSR poll, waits until WIP = 0, and then issues a QIOR burst to refill the line, with no firmware involvement. In XIP mode only the address is given as input.

Both paths share one physical QSPI engine (`qspi_flash_controller`, one TX FIFO, and one RX FIFO). An ownership mux selects, cycle by cycle, whether that engine's inputs come from the APB register bank or from the XIP engine's register bank. `xip_path_busy` is the select. On a conflict, APB wins: the AHB slave holds off an XIP fetch while `indirect_path_busy`, or the RX-drain settle guard, is asserted.

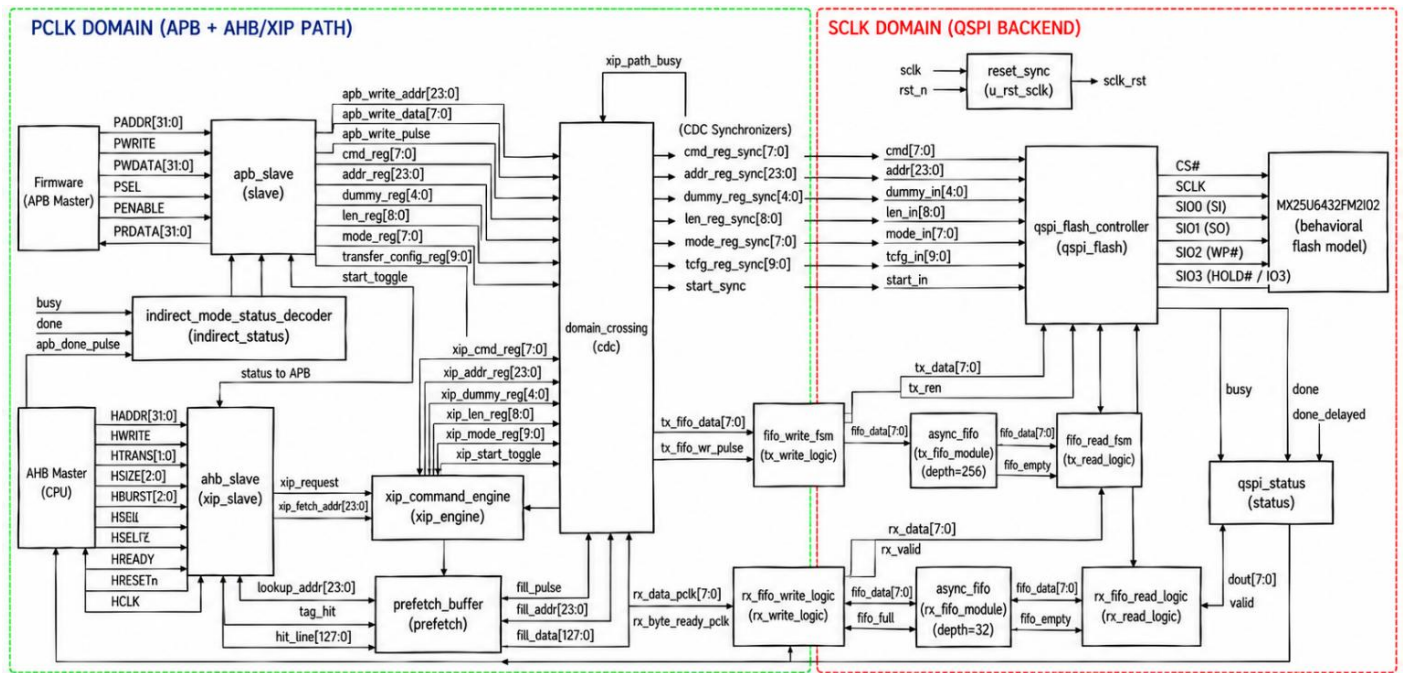


Figure 1 – Microarchitecture of the QSPI Flash Controller. The indirect/APB path and the XIP/AHB path share domain crossing, the TX and RX FIFOs, and the QSPI Flash Controller. The QSPI Flash Controller drives the Macronix MX25U6432FM2I02.

### 3. Interfaces

The table lists every top-level port of top\_cdc\_fifo\_integrated, one signal per row, in the order given in the project document. Ports of the internal modules are not repeated here. pclk is the clock for the APB and AHB-Lite ports. sclk is the SPI-side clock. presetn is the asynchronous active-low reset for both domains.

| Signal Name    | Direction | Width  | Description                      |
|----------------|-----------|--------|----------------------------------|
| sclk           | Input     | 1      | SPI-side clock                   |
| psel           | Input     | 1      | APB select                       |
| penable        | Input     | 1      | APB enable                       |
| pclk           | Input     | 1      | APB/AHB clock                    |
| presetn        | Input     | 1      | Asynchronous active-low reset    |
| pwrite         | Input     | 1      | APB write                        |
| paddr          | Input     | [31:0] | APB address                      |
| pwwdata        | Input     | [31:0] | APB write data                   |
| pready         | Output    | 1      | APB ready                        |
| pslverr        | Output    | 1      | APB error. Tied to 0.            |
| cmd_reg_sync   | Output    | [7:0]  | SCLK-domain command register     |
| len_reg_sync   | Output    | [8:0]  | SCLK-domain length register      |
| dummy_reg_sync | Output    | [4:0]  | SCLK-domain dummy-cycle register |
| addr_reg_sync  | Output    | [23:0] | SCLK-domain address register     |
| start_toggle   | Output    | 1      | PCLK-side start toggle           |
| start_out      | Output    | 1      | SCLK-side start pulse            |

| Signal Name            | Direction | Width  | Description                            |
|------------------------|-----------|--------|----------------------------------------|
| busy_reg               | Output    | 1      | Equal to indirect_path_busy            |
| rx_read_data           | Output    | [7:0]  | RX byte to firmware or to the XIP path |
| cs                     | Output    | 1      | Flash chip-select                      |
| si                     | Output    | 1      | QSPI SIO0                              |
| so                     | Output    | 1      | QSPI SIO1                              |
| wp                     | Output    | 1      | QSPI SIO2                              |
| sio3                   | Output    | 1      | QSPI SIO3                              |
| transmission_completed | Output    | 1      | Indirect-mode status                   |
| indirect_path_busy     | Output    | 1      | Indirect-mode status                   |
| hsel                   | Input     | 1      | AHB select                             |
| haddr                  | Input     | [31:0] | AHB address                            |
| hwrite                 | Input     | 1      | AHB write                              |
| hready                 | Input     | 1      | AHB ready, looped back                 |
| hsize                  | Input     | [2:0]  | AHB transfer size                      |
| hburst                 | Input     | [2:0]  | AHB burst type                         |
| htrans                 | Input     | [1:0]  | AHB transfer type                      |
| hwdata                 | Input     | [31:0] | AHB write data                         |
| hreadyout              | Output    | 1      | AHB ready-out                          |
| hresp                  | Output    | 1      | AHB response                           |
| hrdata                 | Output    | [31:0] | AHB read data                          |

At the QSPI Flash Controller boundary the serial pins are bidirectional (si, so, wp, and sio3 are inout). The top-level list above records the direction given for top\_cdc\_fifo\_integrated. The controller is connected to the behavioral flash model MX25U6432FM2I02, whose pins are SCLK, CS (active-low), SI, SO, WP, and SIO3.

## 4. Clocking and Reset

### 4.1 Clocks

hclk and pclk are tied together. The design uses one unified AMBA-side domain, so the APB/AHB arbitration logic does not contain a clock-domain crossing. sclk is a separate, free-running clock.

| Clock              | Frequency | What it drives                                                                                                                   |
|--------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------|
| pclk (hclk = pclk) | 50 MHz    | APB slave, AHB-Lite slave, prefetch buffer, XIP command engine, indirect-mode status, TX FIFO write side, and RX FIFO read side. |
| sclk               | 12.5 MHz  | QSPI Flash Controller, qspi_status, domain crossing, TX FIFO read side, and RX FIFO write side.                                  |

In the current bring-up configuration pclk/sclk = 4:1. The ratio is not enforced in the design. There is no clock-divider register. The frequency on sclk is whatever is fed to the sclk port.

The clock-domain-crossing structures are described as correct for any ratio, but the synchronizer latency margins were sized assuming sclk is meaningfully slower than pclk. The delayed-done path uses a fixed

4-cycle delay to cover the RX write latch, the FIFO write, and the Gray-pointer register. Running sclk close to, or faster than, pclk is not a supported configuration.

CPOL and CPHA are fixed by construction. SI and SO are driven from the QSPI Flash Controller's registered shift state on sclk edges. There is no register for SPI mode selection.

## 4.2 Reset

There is a single active-low asynchronous reset, presetn. It is synchronized independently into each clock domain by two reset\_sync instances, u\_rst\_pclk and u\_rst\_sclk. Each synchronizer is a 2-flip-flop stage with asynchronous assertion and synchronous deassertion, producing presetn\_pclk and presetn\_sclk.

No sequencing is required between the two domain resets. Both synchronizers free-run from the same presetn and from their own domain clock. Deassertion of presetn\_pclk and presetn\_sclk is not aligned to a defined edge relationship, and software or integration logic should not assume one.

The behavioral flash model does not accept the first command until tVSL = 800 microseconds after power-up or reset. The first APB or AHB transaction has to wait until that window has elapsed after presetn is deasserted. Otherwise the model flags an error and refuses the command.

presetn is not currently synchronized into the asynchronous FIFOs themselves. The FIFO instances take the domain reset directly. Reset during an in-flight indirect-mode or XIP transaction is not a characterized operating case.

## 4.3 Clock-domain crossings

Control and configuration cross from PCLK to SCLK. Status and received data cross from SCLK to PCLK. Each crossing below is the structure named in the project document.

| # | Signal(s)                                                                | Direction    | Structure                                                                                             | Location                       |
|---|--------------------------------------------------------------------------|--------------|-------------------------------------------------------------------------------------------------------|--------------------------------|
| 1 | start (indirect-mode kickoff)                                            | PCLK → SCLK  | Toggle flop (start_toggle) + 3-FF synchronizer chain + XOR edge detector                              | domain_crossing.v              |
| 2 | xip_start_toggle, muxed with the indirect-mode start before the crossing | PCLK → SCLK  | Same toggle + 3-FF + XOR structure. The mux is in the top-level ownership logic, before the crossing. | domain_crossing.v              |
| 3 | qspi_done (transaction-complete pulse)                                   | SCLK → PCLK  | 2-FF synchronizer + edge detect                                                                       | indirect_mode_status_decoder.v |
| 4 | qspi_busy                                                                | SCLK → PCLK  | 2-FF synchronizer (sync1 / qspi_busy_sync)                                                            | apb_slave.v                    |
| 5 | TX FIFO write pointer (firmware preload)                                 | PCLK → SCLK  | Gray-coded pointer, dual-rank synchronized                                                            | async_fifo ADDR_WIDTH=8 (TX)   |
| 6 | RX FIFO write pointer (flash read data)                                  | SCLK → PCLK  | Gray-coded pointer, dual-rank synchronized                                                            | async_fifo ADDR_WIDTH=5 (RX)   |
| 7 | presetn                                                                  | Async → PCLK | 2-FF, async assert / sync deassert                                                                    | reset_sync u_rst_pclk          |

| # | Signal(s) | Direction    | Structure                          | Location              |
|---|-----------|--------------|------------------------------------|-----------------------|
| 8 | presetn   | Async → SCLK | 2-FF, async assert / sync deassert | reset_sync u_rst_sclk |

The start event is a single-cycle pulse in the PCLK domain, and PCLK is faster than SCLK, so a level synchronizer can miss it. The pulse is converted to a level change with a T flip-flop. That toggle is synchronized into the SCLK domain and XORed with its previous value. The XOR is 1 for one SCLK cycle and is the SCLK-domain start pulse. The block description of this synchronizer calls it a dual-rank synchronizer plus a 1-cycle previous-value register. The crossing inventory calls the same structure a 3-FF synchronizer chain.

On the recovered SCLK start pulse, domain\_crossing latches cmd, addr, dummy, len, mode, and transfer\_config into the \_sync registers and asserts start\_out for exactly one SCLK cycle. This is the only place the control and configuration registers cross domains. Figure 7 in Section 6.8 shows an indirect-mode register set being latched on that pulse.

The toggle synchronizer needs roughly 3 SCLK cycles to register a transition. A second start\_toggle transition before the first has propagated through the synchronizer can be missed or coalesced. With SCLK four times slower than PCLK, back-to-back starts at the minimum PCLK spacing are inside that window. The XIP engine issues RDSR and then QIOR as two starts, so this spacing applies to that sequence as well.

## 5. Register Map

Firmware programs the indirect path through the APB register bank. The offset is paddr[3:0]. Unless the table says otherwise, a write is discarded while indirect\_path\_busy is high. pready is 1 in every APB state. pslverr is tied to 0, including for the undefined offsets 0xA–0xF, so a rejected or undefined access still completes the APB handshake with no error.

| Offset | Register Name      | Width | Access | Reset Value | Description                                                                                                                                                           |
|--------|--------------------|-------|--------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x0    | command_code       | 8     | RW     | 0           | Opcode byte sent in the QSPI opcode phase. Write is gated by ~indirect_path_busy.                                                                                     |
| 0x1    | address_bytes      | 24    | RW     | 0           | Flash address for a transaction whose address phase is enabled. Write is gated by ~indirect_path_busy. Four-byte addressing is future scope; see Section 10.          |
| 0x2    | dummy_cycles_count | 5     | RW     | 0           | Dummy SCLK cycle count for the command. Write is gated by ~indirect_path_busy. The programmed value is one less than the datasheet dummy-cycle count; see Section 10. |

| Offset | Register Name                 | Width | Access | Reset Value | Description                                                                                                                                                                                                                                                      |
|--------|-------------------------------|-------|--------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x3    | length_of_data                | 9     | RW     | 0           | Number of bytes written to or read from the flash. Write is gated by ~indirect_path_busy.                                                                                                                                                                        |
| 0x4    | control_signals               | 2     | W      | 0           | LSB (ctrl_reg[0]) starts transmission and clears itself. MSB is abort, which is future scope. A write is accepted only when indirect_path_busy, start_pending, and xip_path_busy are all low. The low-to-high edge of the start bit is what starts the transfer. |
| 0x5    | write_fifo_data               | 8     | W      | 0           | Data byte to store in the TX FIFO. Not gated by indirect_path_busy. Each write is pushed into the TX FIFO. Firmware writes the page bytes here one by one before starting the command.                                                                           |
| 0x6    | read_fifo_data                | 8     | R      | 0           | Next byte from the RX FIFO during a read. A read of this offset clears rx_data_valid.                                                                                                                                                                            |
| 0x7    | transfer_config_reg           | 10    | RW     | 0           | Phase-sequencer control. The QSPI Flash Controller is opcode-blind and follows this register. Write is gated by ~indirect_path_busy. Bit fields are in the next table.                                                                                           |
| 0x8    | qspi_transfer_status_register | 3     | R      | 0           | {transmission_completed, indirect_path_busy, rx_data_valid}                                                                                                                                                                                                      |
| 0x9    | mode_byte                     | 8     | RW     | 0           | M7–M0 mode byte for the flash transaction. The value depends on the behavioral flash model. Write is gated by ~indirect_path_busy.                                                                                                                               |

### 5.1 transfer\_config\_reg bit fields

transfer\_config\_reg[9:0] selects which phases run and how many I/O lines each phase uses. A value of 11 on a line-width field is not defined in the project document.

| Bit Range | Field Name | Values | Description |
|-----------|------------|--------|-------------|
|-----------|------------|--------|-------------|

| Bit Range | Field Name            | Values                                   | Description                                                                                                                                  |
|-----------|-----------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| [9:8]     | data_phase_line_width | 00 = 1-wire, 01 = 2-wire,<br>10 = 4-wire | I/O width during the data phase.                                                                                                             |
| [7:6]     | addr_phase_line_width | 00 = 1-wire, 01 = 2-wire,<br>10 = 4-wire | I/O width during the address, mode, and dummy phases.                                                                                        |
| [5]       | addr_byte_count       | 0 = 3-byte address, 1 = 4-byte address   | Selects a 24-bit or 32-bit address field. The current design supports 3-byte addresses only, because of the prefetch buffer. See Section 10. |
| [4]       | data_direction        | 0 = read from flash, 1 = write to flash  | Direction of the data phase.                                                                                                                 |
| [3]       | addr_phase_enable     | 0 = skip, 1 = include                    | Whether an address phase follows the opcode.                                                                                                 |
| [2]       | data_phase_enable     | 0 = skip, 1 = include                    | Whether a data phase is present.                                                                                                             |
| [1]       | mode_phase_enable     | 0 = skip, 1 = include                    | Whether a mode-byte phase follows the address phase.                                                                                         |
| [0]       | dummy_phase_enable    | 0 = skip, 1 = include                    | Whether a dummy-cycle phase is present.                                                                                                      |

## 5.2 Firmware use of the bank

Firmware writes the registers one by one and then starts the transfer by setting the start bit in control\_signals. For a Read Status Register, firmware writes 0x05 into command\_code, programs the other registers the command needs, and asserts start.

While the indirect path or the XIP path is busy, a firmware write into the APB register bank is discarded, except for write\_fifo\_data, which stays writable so firmware can stream TX bytes. Firmware watches the Transmission Completed flag. For a read, the received byte is presented in read\_fifo\_data after the fourth clock cycle.

Setting the flash Quad Enable (QE) bit is firmware's job, and it is done once during initialization. Writing the status register with the Write Status Register command asserts the flash WIP bit and takes 40 ms (tW, from the MX25U6432FM2I02 datasheet/model). The XIP path can perform QIOR only when QE is already set. Re-asserting QE on every transaction would add that 40 ms every time, so firmware sets QE once and leaves it set for the session.

A write to a busy-gated register still gets pready = 1 and pslverr = 0. The write is silently dropped. Two alternatives were considered and not implemented: stall pready (which can insert a very long wait, for example during a tBE = 300 ms block erase) or assert pslverr on the gated cycle. The design does neither.

## 6. Functional Description

This section describes how a transfer starts, how data moves, and how it completes. The two host paths share one serial engine. The ownership mux, the FIFOs, and the prefetch line are the pieces that keep those paths from using that engine at the same time or from returning stale data.



## 6.1 Overall architecture

The indirect path is the firmware command engine. The XIP path is the AHB-Lite read path. Only one of them drives qspi\_flash\_controller at a time. domain\_crossing receives a request from either path, keeps the priority, and sends the selected command set to the QSPI Flash Controller. Bytes going out to the flash travel through the TX FIFO. Bytes coming back travel through the RX FIFO to the APB read register or to the prefetch buffer.

The modules below are the design files named in the project document.

| Module                                      | Role                                                                                                                                                                            |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| command_code.h                              | Register offsets, command opcodes, bit-field definitions, and the fixed XIP presets.                                                                                            |
| reset_sync.v                                | Asynchronous assertion and synchronous deassertion of reset in each clock domain.                                                                                               |
| domain_crossing.v                           | Toggle-based PCLK-to-SCLK crossing of the selected command set. Sends the selected command to the QSPI Flash Controller.                                                        |
| async_fifo.v                                | Dual-clock FIFO using Gray-coded read and write pointers. Shared by the TX and RX paths.                                                                                        |
| fifo_write_fsm.v                            | TX FIFO write control from APB FIFO writes.                                                                                                                                     |
| fifo_read_fsm.v                             | Reads the TX FIFO and supplies bytes to the QSPI Flash Controller.                                                                                                              |
| rx_fifo_write_logic.v                       | Captures a valid QSPI RX byte and writes the RX FIFO.                                                                                                                           |
| rx_fifo_read_logic.v                        | Reads the RX FIFO and generates rx_byte_ready.                                                                                                                                  |
| qspi_status.v                               | QSPI busy, done, and delayed-done from chip-select.                                                                                                                             |
| qspi_flash_controller.v                     | Serial transaction engine. Drives the pins of the behavioral flash model.                                                                                                       |
| prefetch_buffer.v                           | One 16-byte XIP line. The AHB slave returns the line on a valid tag hit; otherwise the line is fetched and stored. An overlapping indirect write or erase invalidates the line. |
| indirect_mode_status_decoder.v              | Indirect-path busy and transmission-completed status, synchronized into the PCLK domain for the APB status register.                                                            |
| apb_slave.v                                 | APB interface and the indirect-mode register bank.                                                                                                                              |
| ahb_slave.v                                 | AHB-Lite interface and the XIP request.                                                                                                                                         |
| xip_command_engine.v                        | Command and configuration for an autonomous XIP flash read.                                                                                                                     |
| top_module.v /<br>top_cdc_fifo_integrated.v | Top-level integration. See the naming comment in Section 2.                                                                                                                     |
| MX25U6432FM2I02.v                           | Behavioral model of the Macronix flash connected to the serial pins.                                                                                                            |

## 6.2 Indirect / APB path

Firmware fills the register bank and then sets the start flag in control\_signals. Once the transmission has started, the register data goes to domain\_crossing. That module checks whether the flash transfer can be launched and crosses the selected register set to qspi\_flash\_controller. The controller latches the values and drives them to the behavioral flash model on the serial pins.

The APB slave is a three-state access sequence, idle / setup / access, which the project document calls a APB3 FSM. It holds cmd\_reg, addr\_reg, dummy\_reg, len\_reg, transfer\_config\_reg, mode\_data\_reg,



tx\_fifo\_data, and the self-clearing ctrl\_reg[0] start bit. A write of the start bit produces a one-cycle start and toggles start\_toggle, which is the event domain\_crossing carries into the SCLK domain.

The read-only status register mirrors {transmission\_completed, indirect\_path\_busy, rx\_data\_valid}. qspi\_busy is synchronized into this clock domain with two flip-flops (sync1, then qspi\_busy\_sync) before it is used.

Figure 2 shows the register writes for Read Status Register, command code 0x05. Firmware writes the bank and then sets the start bit. Figure 3 shows Page Program, command code 0x02: the data bytes are written to write\_fifo\_data first, the command registers follow, and len\_reg holds the byte count. The illustration uses a 12-byte page.

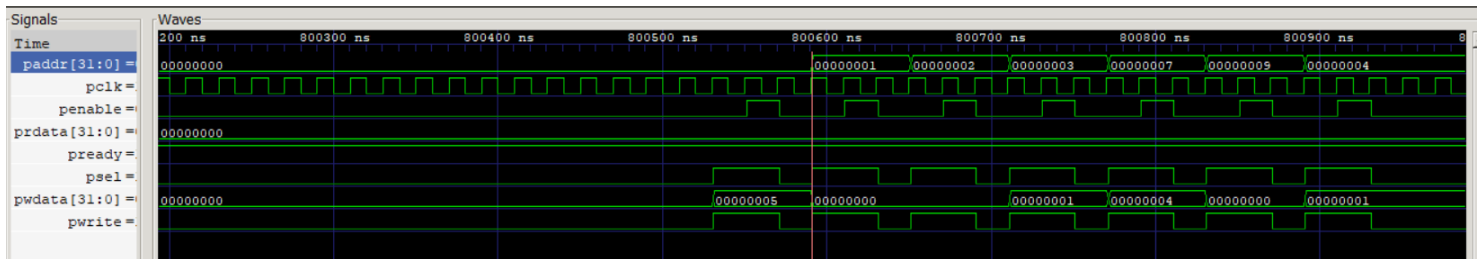


Figure 2 – Indirect-mode APB programming for Read Status Register, command code 0x05. The register bank is written and the start bit in control\_signals is set at the end of the sequence.

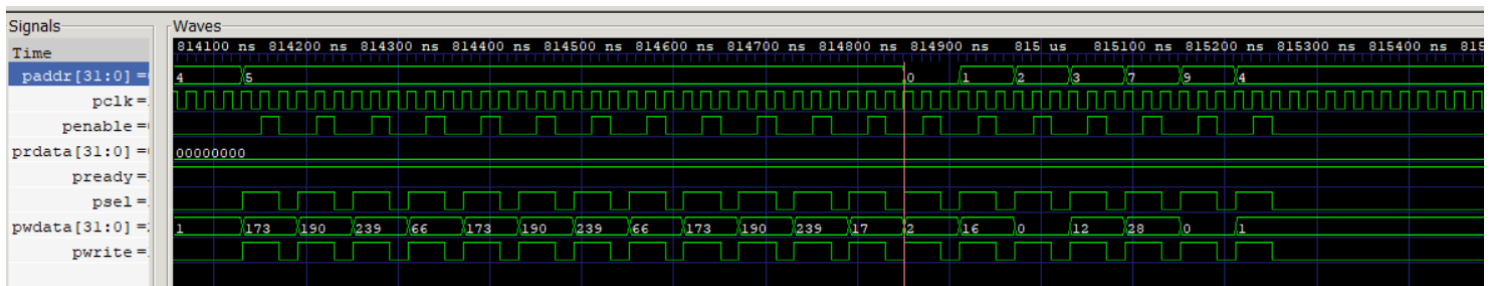


Figure 3 – Indirect-mode APB programming for Page Program, command code 0x02. Data bytes are written to write\_fifo\_data (offset 0x5) before the command registers. len\_reg (offset 0x3) is loaded with the byte count; this illustration loads 12.

### 6.3 XIP / AHB path

An AHB-Lite master presents an address. The AHB slave checks the prefetch buffer. If the requested line is present, that data is returned on hrddata. If it is not, the slave asks the XIP command engine to read the line from the flash. The slave latches haddr, hsize, hburst, and hwrite on the address-phase accept cycle, because AHB-Lite keeps them stable for that cycle only.

The slave is a single-beat NONSEQ-driven machine: idle, then check\_prefetch\_buffer, then data\_tag\_hit on a hit, or wait\_engine, then fetch\_wait, then fill\_prefetch\_buffer on a miss. hreadyout is held off through the miss. HRDATA is a byte, halfword, or word taken from the 128-bit line using haddr\_latched[3:0].

The XIP path is read-only. A write is sent to write\_reject and hresp is asserted. xip\_path\_busy is asserted only in fetch\_wait, which is the time the shared engine is actually being driven by the XIP register bank. The top-level ownership mux uses that signal. wait\_engine does not issue a fetch while indirect\_path\_busy or rx\_drain\_busy is set, so APB keeps priority.

If the prefetch buffer holds a line and a later indirect program or erase covers that same address, the line is invalidated so the next AHB read cannot return the old data. The invalidation rule is in Section 6.14.

Figure 4 shows a cold miss. htrans indicates a new NONSEQ request, the prefetch valid flag is low, and the XIP path therefore fetches the line. It first sends command 0x05 (RDSR). In the illustration the returned byte is 0x42, which the project document describes as WIP low, and the path then performs QIOR. Figure 5 continues that QIOR transfer. Figure 6 shows the 16-byte line being written into the prefetch buffer.

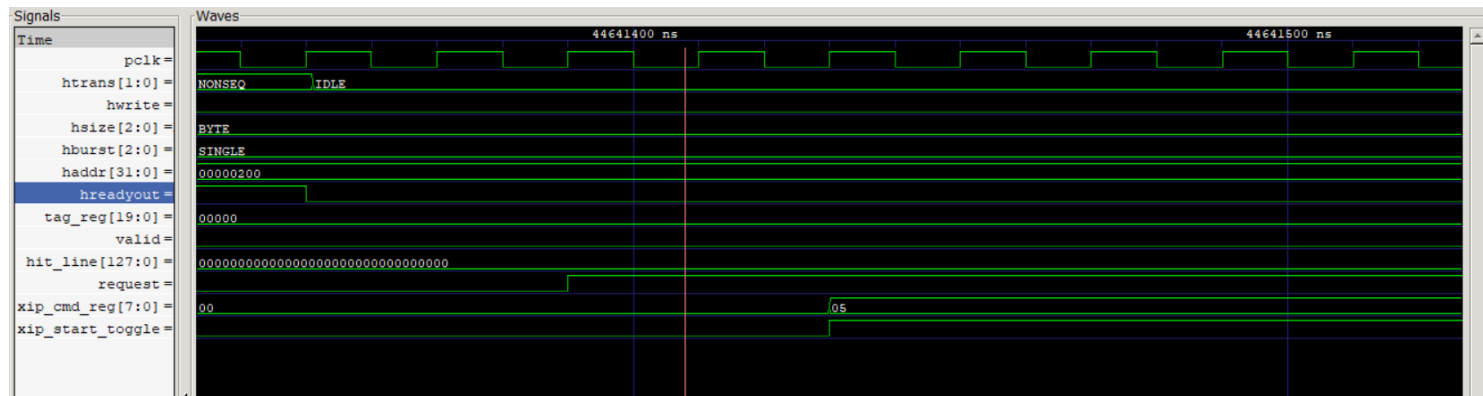


Figure 4 – XIP read request. An AHB-Lite NONSEQ access finds the prefetch line invalid. The XIP path issues Read Status Register (command 0x05) before the data fetch.

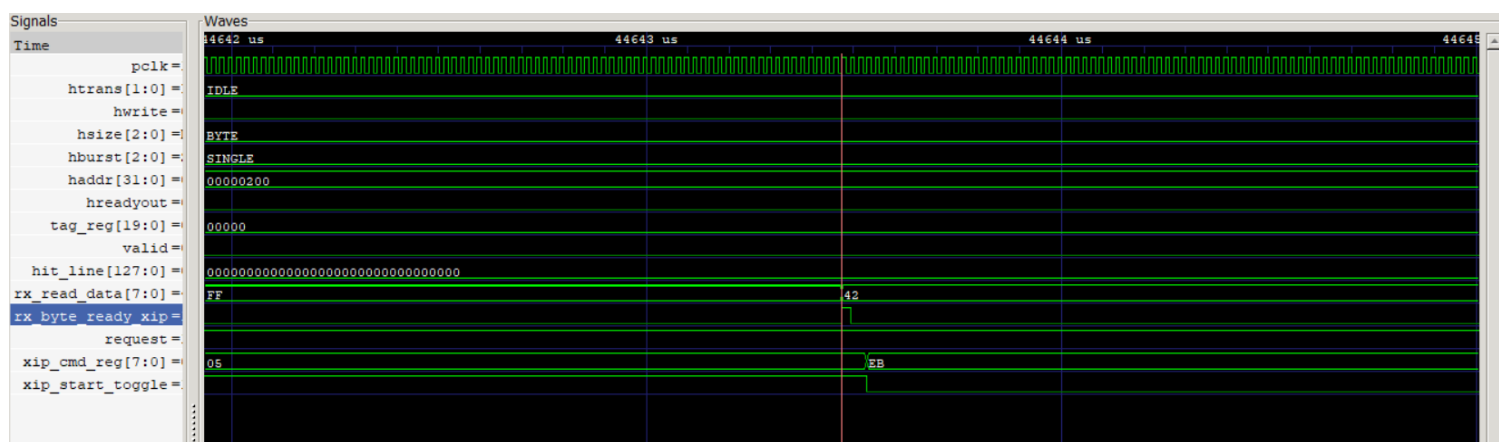


Figure 5 – XIP fetch after the status poll. The illustration's RDSR data is 0x42, described in the source as WIP low, and the path then runs Quad I/O Read.

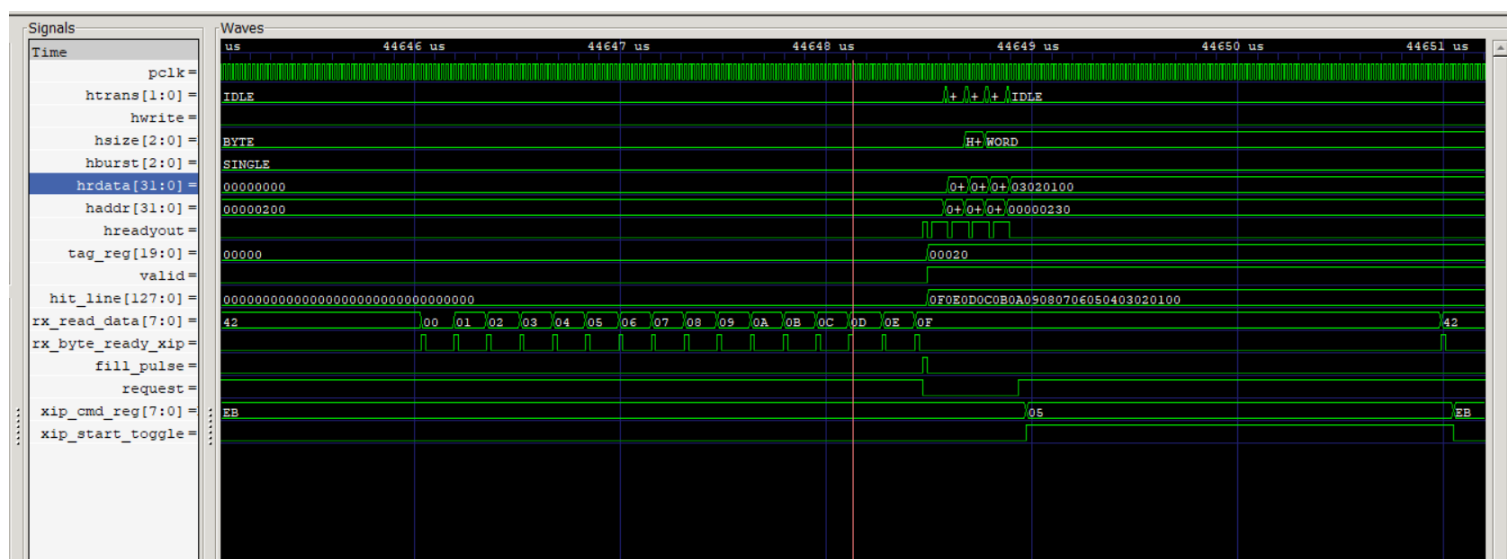


Figure 6 – End of an XIP line fill. After 16 bytes have been read, fill\_pulse writes the line into the prefetch buffer and the valid flag is asserted. The AHB-Lite slave then returns the addressed bytes on hrdata.

The corresponding serial-pin transactions are shown in Section 6.9.

#### 6.4 APB register bank

apb\_slave is the programmer's model for the indirect path. The registers, their gating, and the start bit are specified in Section 5. Register writes are blocked while indirect\_path\_busy or xip\_path\_busy is high, except write\_fifo\_data. start is the rising edge of ctrl\_reg[0]. start\_toggle is the toggle flop that carries that edge into domain\_crossing.

#### 6.5 AHB-Lite XIP slave

ahb\_slave accepts a NONSEQ access, compares the latched address with the prefetch line, and either returns the hit or requests a line fill. It does not drive the serial engine itself. It raises xip\_request for the XIP command engine and raises xip\_path\_busy only while it is in fetch\_wait. hburst is latched and is not used. A write never reaches the flash.

#### 6.6 Prefetch buffer

prefetch\_buffer is a single 16-byte line, direct mapped. The tag is addr[23:4]. tag\_hit is a combinational compare of the valid tag against the lookup address. The parameters are LINE\_SIZE = 16, ADDR\_WIDTH = 24, and OFFSET\_BITS = 4.

A completed XIP QIOR fills the line with fill\_pulse, fill\_addr, and fill\_data. invalidate\_pulse clears the line. The top level asserts invalidate\_pulse when a completed APB write-class command (program or erase) overlaps the cached line, so XIP does not serve stale data after firmware programs or erases the region that is currently cached.

#### 6.7 XIP command engine

xip\_command\_engine is the autonomous WIP-poll and QIOR sequencer. Its states are idle, issue\_rdsr, wait\_rdsr, issue\_qior, wait\_qior, and done. From wait\_rdsr it returns to issue\_rdsr while WIP is set; otherwise it goes to issue\_qior. It has its own register bank (xip\_cmd\_reg, xip\_addr\_reg, xip\_dummy\_reg, xip\_len\_reg, xip\_tcfg\_reg, xip\_mode\_reg, and xip\_start\_toggle). The top-level mux substitutes that bank for the APB registers whenever xip\_path\_busy is high.

Completion is a byte count, not a watch of transmission\_completed. The engine counts rx\_byte\_ready\_xip pulses: 1 for RDSR and 16 for QIOR. MAX\_WIP\_POLLS defaults to 65535 and bounds the RDSR loop. If WIP never clears, the engine raises timeout\_error instead of polling forever. The numerical margin of that default against worst-case flash timing is not established in the project document; see Section 10.

After an indirect erase, the flash keeps WIP high for a model-dependent time after the controller has finished the command. The project document describes that time as around 300ms for a block erase. The XIP path is responsible for polling WIP: it issues Read Status Register, and only when WIP is low does it start the QIOR. Otherwise it keeps issuing RDSR.

#### 6.8 Domain crossing

domain\_crossing is the PCLK-to-SCLK control-register crossing. Its states are idle, latch\_inputdata, and send\_data. The start toggle is synchronized as described in Section 4.3. On the recovered edge the FSM latches the command set into the \_sync registers and pulses start\_out for one SCLK cycle.

The module takes requests from both the indirect path and the XIP path and forwards one of them according to priority. The project document describes an ownership mux in the top level that selects, cycle by cycle, whether domain\_crossing sees the APB register bank or the XIP register bank. The crossing

inventory states that this mux happens before the crossing, and that both start toggles then use the one synchronizer in domain\_crossing.

Figure 4 shows an indirect-mode request crossing. The APB registers change as firmware writes them, one register at a time. When domain\_crossing receives the start pulse it latches that command set.

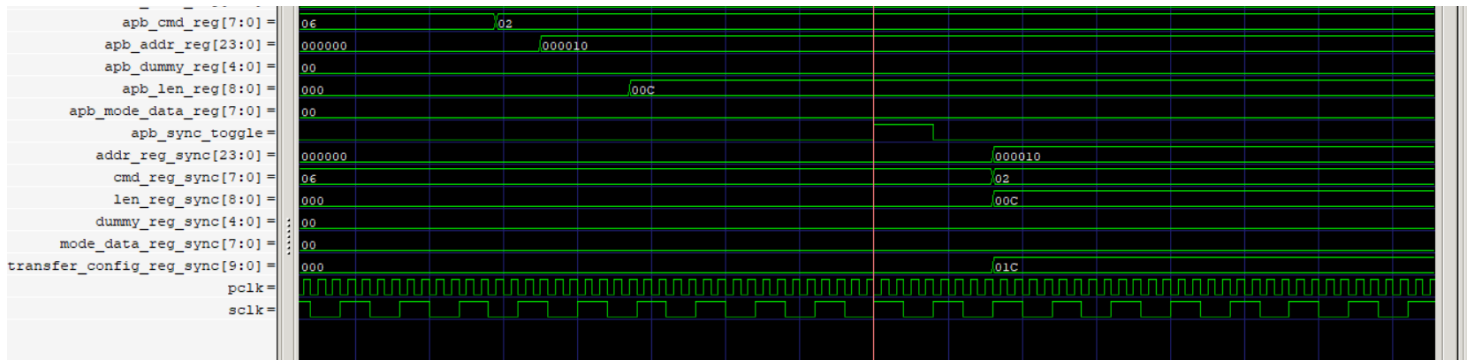


Figure 7 – Indirect-mode command set crossing into the SCLK domain. The APB registers update as firmware writes them. domain\_crossing latches the set when the start toggle is received.

### 6.9 QSPI Flash Controller

qspi\_flash\_controller is the SPI/QSPI protocol engine in the SCLK domain. It is a generic phase sequencer, not a per-opcode case statement. The phases are opcode, address, mode, dummy, and data. Which phases run, and whether a phase uses 1, 2, or 4 I/O lines, comes entirely from transfer\_config\_reg[9:0]. The engine drives cs, si, so, wp, and sio3. On a read it produces dout and valid for each received byte. On a write it pulses tx\_data\_req one cycle before the TX byte is latched, which covers the TX FIFO's registered read latency.

The command, address, and configuration arrive from domain\_crossing. Write data arrives from the TX FIFO. Read data leaves through the RX FIFO toward the APB read register or the prefetch buffer. The serial pins connect to the behavioral flash model MX25U6432FM2I02.

Chip-select is inactive while the engine is idle and active through the phases of a transfer. The opcode is always shifted on a single line. Address, mode, dummy, and data follow only when the corresponding enable bit is set, in that order.

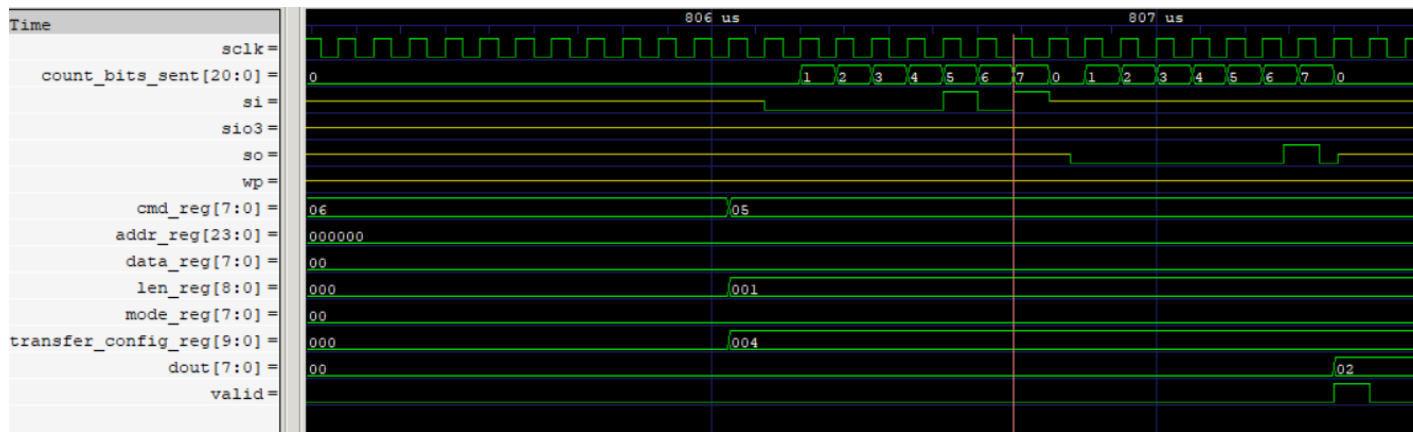


Figure 8 – QSPI Flash Controller transaction for Read Status Register, command 0x05. The opcode is shifted on si. The flash returns the status byte on so. valid is asserted when the byte is presented on dout for the RX FIFO.

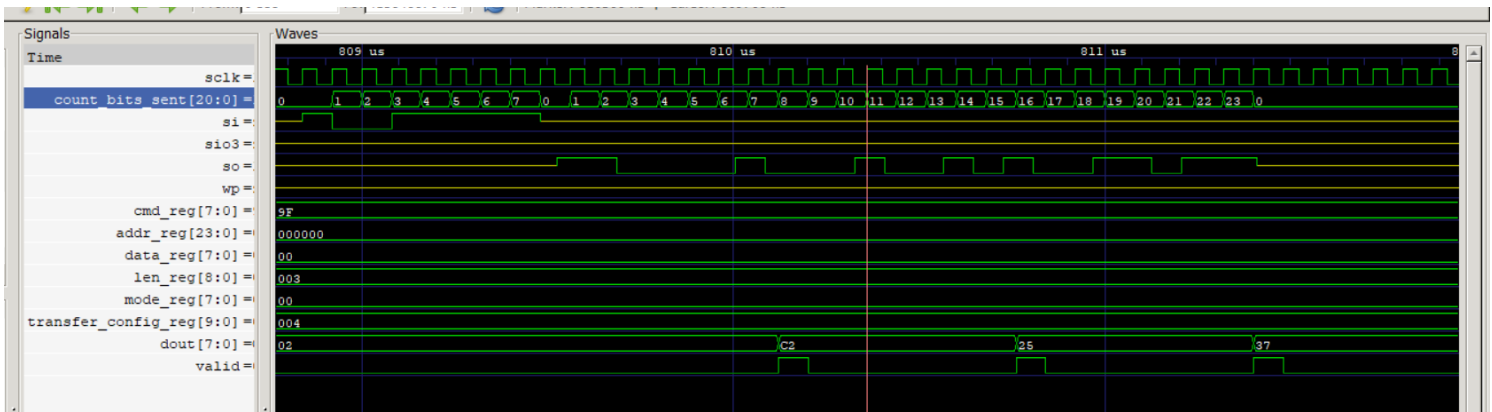


Figure 9 – QSPI Flash Controller transaction for Read JEDEC ID, command 0x9F.

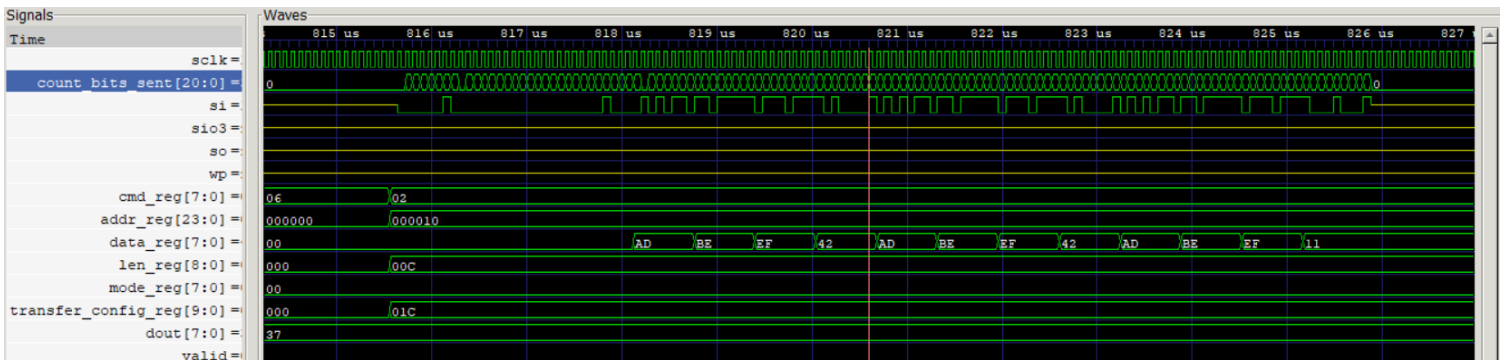


Figure 10 – QSPI Flash Controller transaction for Page Program, command 0x02. The illustration writes 12 bytes. data\_reg[7:0] is the byte read from the TX FIFO that firmware loaded.

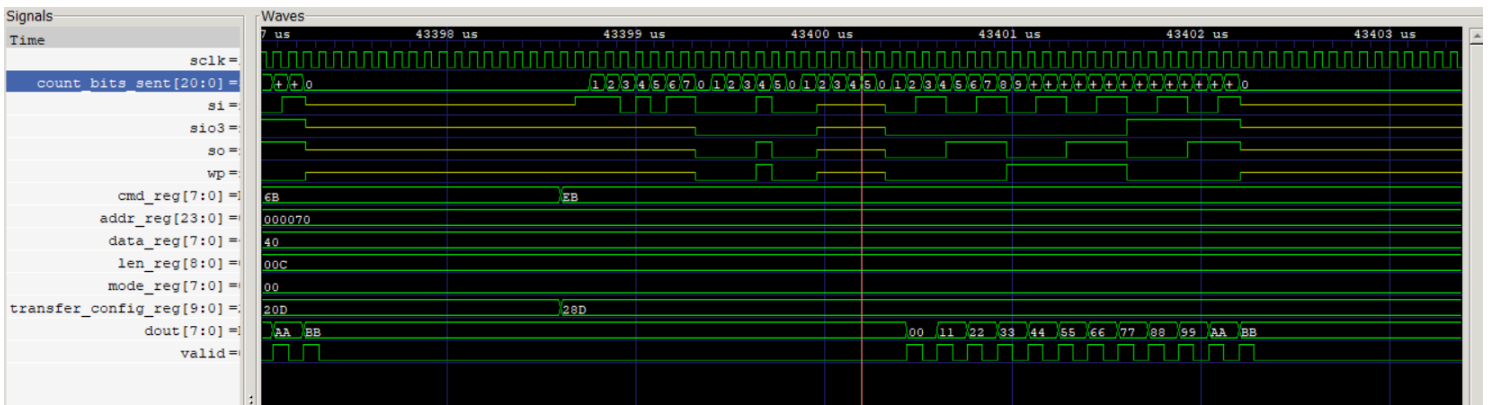


Figure 11 – QSPI Flash Controller transaction for Quad I/O Read (QIOR), command 0xEB.

The project document's captured log also says the mode phase was a no-op because mode\_reg was tied to a constant 0 and there was no APB path for the mode byte yet. The register map in the same document defines mode\_byte at offset 0x9 as a read/write register, and the APB slave description includes mode\_data\_reg in the programmer's model.

## 6.10 TX FIFO path

The TX path carries firmware data to the flash. fifo\_write\_fsm, in the PCLK domain, pulses wen one cycle after tx\_fifo\_wr\_pulse so that tx\_fifo\_data has settled. The TX async\_fifo is 256 deep (ADDR\_WIDTH = 8). Its write side is pclk and its read side is sclk. The depth is 256 because the largest page program described for this design is 256 bytes.

fifo\_read\_fsm, in the SCLK domain, asserts ren when the transfer is a write, start is high, and byte\_req (tx\_data\_req from the flash controller) fires. It counts bytes sent against len\_reg. During the data phase the controller takes one byte at a time from the FIFO and shifts it to the flash.

The intended use is to preload the FIFO completely before issuing start. write\_fifo\_data is not gated by indirect\_path\_busy, so a write during a transaction is still accepted. The TX FIFO is a single preload; the design does not describe a mid-burst refill.

### 6.11 RX FIFO path

The RX path carries flash data back to firmware or to the XIP engine. rx\_fifo\_write\_logic, in the SCLK domain, latches dout and writes the FIFO with a one-cycle wen when valid is asserted. The RX async\_fifo is 32 deep (ADDR\_WIDTH = 5). Its write side is sclk and its read side is pclk.

rx\_fifo\_read\_logic, in the PCLK domain, drains the FIFO while the transfer is a read, the FIFO is not empty, and the number of bytes already read is still below that transfer's len\_reg. The drain is armed by the transfer's start pulse and is cleared when the count reaches len\_reg. Bytes can therefore leave the FIFO while the serial transfer is still running; the drain does not wait for the transfer to finish. A ren\_d pipeline stage keeps read\_data from sampling the FIFO's registered output one cycle early.

data\_direction into this block is the muxed transfer configuration (the value selected for the owner of the current transfer), not the raw APB transfer\_config\_reg. Using the raw APB value lets the XIP path's RDSR poll stall.

At the top, rx\_byte\_ready is split. rx\_byte\_ready\_apb is rx\_byte\_ready AND NOT xip\_path\_busy. rx\_byte\_ready\_xip is rx\_byte\_ready AND xip\_path\_busy. The APB slave and the XIP engine therefore do not see each other's captured bytes on the shared RX chain.

The concurrent drain is sized so the 32-entry RX FIFO does not have to hold a full 256-byte read. A drain that stops at the moment the FIFO is full can drop or repeat a byte. That boundary is a property of this depth, not a second RX buffer.

### 6.12 QSPI status and completion

qspi\_status tracks chip-select in the SCLK domain. The states are idle, qspi\_operation while cs is low, and qspi\_done when cs returns high, then idle. busy and done follow those states. done is the immediate completion pulse and feeds indirect\_mode\_status\_decoder. done\_delayed is the same pulse held for a fixed 4 cycles and feeds only the RX read logic. The four cycles cover the RX write latch, the FIFO write, and the Gray-pointer register, so the write pointer is visible on the read side before the drain starts pulling.

### 6.13 Indirect-mode status

indirect\_mode\_status\_decoder produces the PCLK-domain busy and done flags for the APB slave and for the top-level arbitration and invalidation logic. It dual-rank synchronizes qspi\_done from the SCLK domain into pclk and edge-detects it. The states are idle, transmission\_begun when start is seen, and transmission\_completed\_by\_qspi when the synchronized done pulse arrives, then idle. transmission\_completed and indirect\_path\_busy are the outputs written into qspi\_transfer\_status\_register.

### 6.14 Cache invalidation

The top level watches indirect writes that can make the cached XIP line stale. On the start pulse of a write-class APB command (a command whose data direction is a write and whose address phase is



enabled), it latches the command address. When that command completes (apb\_write\_done\_pulse), it compares the latched address with the cached tag.

The compare is erase-aware. A sector erase and a block erase use a range mask so the whole erased region is compared with the cached line, not only the exact address. A page program compares the 16-byte line. If the cached line is valid and overlaps, invalidate\_pulse clears the prefetch buffer. A program or erase that does not overlap the cached line leaves the line valid.

### 6.15 Arbitration between the APB and XIP paths

APB wins on a conflict. ahb\_slave stays in wait\_engine while indirect\_path\_busy or rx\_drain\_busy is asserted, and it does not issue an XIP fetch during that time. There is no round-robin and no other fairness bound. Continuous back-to-back indirect transactions can therefore hold the XIP path off for as long as the APB traffic continues.

indirect\_path\_busy drops when the QSPI engine reports done, which is when chip-select is deasserted. The RX FIFO can still be draining the last APB bytes for several PCLK cycles after that. The RX-drain settle guard closes that window. rx\_drain\_settle holds the AHB slave off until the RX FIFO has been empty for 3 consecutive PCLK cycles. Without that guard, an XIP start would reset the RX byte count in the middle of an APB read and truncate it.

The same owner select steers the shared RX drain: the length, the start pulse, and the data direction presented to rx\_fifo\_read\_logic come from the XIP bank when xip\_path\_busy is high and from the APB bank otherwise.

---

## 7. FSM / State Transitions

The tables below use the state names given in the project document. A condition that the document does not state is left as not specified, rather than filled in from a reading of the RTL.

### 7.1 APB slave

The project document names a three-state APB sequence: idle, setup, and access. It does not enumerate the cycle-by-cycle conditions. psel and penable are the APB select and enable inputs. The start bit is a separate, self-clearing flag inside access and is not an extra state.

| State  | Description                                                                                                                                       | Transitions To | Condition                                                    |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------|--------------------------------------------------------------|
| idle   | No APB access in progress.                                                                                                                        | setup          | <i>[Not specified in the existing project documentation]</i> |
| setup  | APB setup phase of an access.                                                                                                                     | access         | <i>[Not specified in the existing project documentation]</i> |
| access | APB access phase. A write can update the register bank, subject to the gating in Section 5. A read returns read_fifo_data or the status register. | idle or setup  | <i>[Not specified in the existing project documentation]</i> |



## 7.2 AHB-Lite XIP slave

Single-beat NONSEQ-driven sequence. Address-phase haddr, hsize, hburst, and hwrite are latched on the accepting cycle and then held.

| State                 | Description                                                                                            | Transitions To        | Condition                                                                                 |
|-----------------------|--------------------------------------------------------------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------------|
| idle                  | Waiting for an AHB-Lite request.                                                                       | check_prefetch_buffer | NONSEQ access accepted.                                                                   |
| check_prefetch_buffer | Compare the latched address with the prefetch line. A write is rejected here because XIP is read-only. | write_reject          | The latched access is a write.                                                            |
| check_prefetch_buffer | Compare the latched address with the prefetch line.                                                    | data_tag_hit          | tag_hit.                                                                                  |
| check_prefetch_buffer | Compare the latched address with the prefetch line.                                                    | wait_engine           | Read, and the tag misses.                                                                 |
| write_reject          | Write to the XIP region. hresp is asserted.                                                            | idle                  | Response has been given.                                                                  |
| data_tag_hit          | Return byte, halfword, or word from the 128-bit line on hrdata.                                        | idle                  | The access completes without a new request.                                               |
| wait_engine           | Hold the fetch off so the indirect path keeps the shared engine.                                       | wait_engine           | indirect_path_busy or rx_drain_busy.                                                      |
| wait_engine           | Hold the fetch off so the indirect path keeps the shared engine.                                       | fetch_wait            | Both busy conditions are clear.                                                           |
| fetch_wait            | xip_path_busy is asserted. The XIP command engine owns the shared engine and runs RDSR, then QIOR.     | fill_prefetch_buffer  | The engine reports done, or timeout_error. A stuck WIP must not hold the AHB bus forever. |
| fill_prefetch_buffer  | Write the fetched 16-byte line into the prefetch buffer with fill_pulse.                               | check_prefetch_buffer | The line has been presented to the buffer.                                                |

### 7.3 XIP command engine

Autonomous sequencer in the PCLK domain. It counts rx\_byte\_ready\_xip pulses: one byte ends RDSR, sixteen bytes end QIOR.

| State      | Description                                                                                    | Transitions To | Condition                                                                                     |
|------------|------------------------------------------------------------------------------------------------|----------------|-----------------------------------------------------------------------------------------------|
| idle       | Waiting for a line request from the AHB slave.                                                 | issue_rdsr     | request is asserted.                                                                          |
| issue_rdsr | Load the RDSR command into the XIP register bank and toggle xip_start_toggle.                  | wait_rdsr      | The RDSR command has been issued.                                                             |
| wait_rdsr  | Wait for the one status byte. WIP is taken from that byte.                                     | issue_rdsr     | The status byte has arrived and WIP is set, and the poll count is still inside MAX_WIP_POLLS. |
| wait_rdsr  | Wait for the one status byte.                                                                  | issue_qior     | The status byte has arrived and WIP is clear.                                                 |
| wait_rdsr  | Wait for the one status byte.                                                                  | done           | WIP is still set when the poll count reaches MAX_WIP_POLLS. timeout_error is raised.          |
| issue_qior | Load the QIOR command, the line address, and the XIP transfer configuration, and toggle start. | wait_qior      | The QIOR command has been issued.                                                             |
| wait_qior  | Collect 16 data bytes into the line.                                                           | done           | 16 bytes have been captured. done is a one-cycle pulse and line_data is valid.                |
| done       | Hold the result until the AHB slave drops the request.                                         | idle           | request is deasserted.                                                                        |

### 7.4 Domain crossing

Runs in the SCLK domain. The launch condition is the XOR edge of the synchronized start toggle, after the ownership mux has selected which path's start and registers are presented.

| State | Description | Transitions To | Condition |
|-------|-------------|----------------|-----------|
|-------|-------------|----------------|-----------|

| State           | Description                                                                                       | Transitions To  | Condition                       |
|-----------------|---------------------------------------------------------------------------------------------------|-----------------|---------------------------------|
| idle            | Waiting for a start edge from the indirect path or the XIP path.                                  | latch_inputdata | Synchronized start pulse.       |
| latch_inputdata | Latch cmd, addr, dummy, len, mode, and transfer_config into the _sync registers.                  | send_data       | The latch cycle completes.      |
| send_data       | Assert start_out for exactly one SCLK cycle. This is the start seen by the QSPI Flash Controller. | idle            | That one cycle has been issued. |

## 7.5 QSPI Flash Controller

Phase sequencer in the SCLK domain. A disabled phase is skipped. The line width of each phase is the field in Section 5.1. The document names the phases; it does not give a separate state identifier beyond those phase names.

| State  | Description                                  | Transitions To | Condition                                                           |
|--------|----------------------------------------------|----------------|---------------------------------------------------------------------|
| idle   | Chip-select inactive. Waiting for start_out. | opcode         | start.                                                              |
| opcode | Shift the opcode byte on a single I/O line.  | address        | Opcode phase finished and addr_phase_enable is set.                 |
| opcode | Shift the opcode byte on a single I/O line.  | mode           | Opcode finished, address disabled, mode_phase_enable set.           |
| opcode | Shift the opcode byte on a single I/O line.  | dummy          | Opcode finished, address and mode disabled, dummy_phase_enable set. |
| opcode | Shift the opcode byte on a single I/O line.  | data           | Opcode finished and only the data phase is enabled.                 |
| opcode | Shift the opcode byte on a single I/O line.  | idle           | Opcode finished and no later phase is enabled.                      |

| State   | Description                                                                                                                                  | Transitions To             | Condition                                                                            |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|--------------------------------------------------------------------------------------|
| address | Shift the address on 1, 2, or 4 lines, as programmed.                                                                                        | mode, dummy, data, or idle | Address phase finished. The next enabled phase runs, in the order mode, dummy, data. |
| mode    | Shift the mode byte on the address-phase line width.                                                                                         | dummy, data, or idle       | Mode phase finished. The next enabled phase runs.                                    |
| dummy   | Insert the programmed dummy SCLK cycles.                                                                                                     | data or idle               | Dummy count exhausted. Data runs only if data_phase_enable is set.                   |
| data    | Shift len_reg bytes to the flash, or capture len_reg bytes from the flash, at the programmed data width. A captured byte is dout with valid. | idle                       | The programmed number of bytes has been transferred.                                 |

## 7.6 TX and RX FIFO control

The project document describes the FIFO blocks by their pulses and counts. It does not give them state names. Those names are *[Not specified in the existing project documentation]*. The behavior is in Sections 6.10 and 6.11.

- fifo\_write\_fsm pulses wen one PCLK cycle after tx\_fifo\_wr\_pulse, once tx\_fifo\_data has settled.
- fifo\_read\_fsm asserts ren while the transfer is a write, start is high, and tx\_data\_req fires, and it counts bytes against len\_reg.
- rx\_fifo\_write\_logic writes the RX FIFO for one cycle when valid is asserted, latching dout.
- rx\_fifo\_read\_logic has no named states in the project document. It drains while the transfer is a read, the FIFO is not empty, and the byte count is below len\_reg. The count resets on the transfer start pulse.

## 7.7 QSPI status

SCLK-domain status derived from chip-select. done is immediate. done\_delayed is the 4-cycle delayed copy used only by the RX drain.

| State          | Description                                | Transitions To | Condition        |
|----------------|--------------------------------------------|----------------|------------------|
| idle           | No serial transfer. busy and done are low. | qspi_operation | cs is low.       |
| qspi_operation | Chip-select is active. busy is high.       | qspi_done      | cs returns high. |

| State     | Description                                | Transitions To | Condition                  |
|-----------|--------------------------------------------|----------------|----------------------------|
| qspi_done | One cycle of completion. done is asserted. | idle           | The completion cycle ends. |

### 7.8 Indirect-mode status decoder

PCLK-domain status for the APB bank and for arbitration. qspi\_done is synchronized with two flip-flops and then edge-detected before it is used.

| State                          | Description                                                                                                    | Transitions To                 | Condition                               |
|--------------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------|-----------------------------------------|
| idle                           | No indirect transfer in progress. transmission_completed is the idle indication and indirect_path_busy is low. | transmission_begun             | start.                                  |
| transmission_begun             | Indirect transfer in progress. indirect_path_busy is high and transmission_completed is low.                   | transmission_completed_by_qspi | Synchronized qspi_done pulse.           |
| transmission_completed_by_qspi | The serial engine has completed. transmission_completed is high and indirect_path_busy is low.                 | idle                           | The completion state has been reported. |

reset\_sync and the asynchronous FIFO are not state machines of this kind. reset\_sync is the 2-flip-flop reset synchronizer in Section 4.2. The FIFO is the Gray-pointer dual-clock buffer in Section 4.3. The prefetch buffer is a single valid line, not a multi-state machine; its fill and invalidate behavior is in Section 6.6.

## 8. Reference Specification and Supported Features

The flash named by the project document is the Macronix MX25U6432FM2I02 QSPI NOR flash. The document gives timing taken from that device's datasheet and behavioral model (tW, tVSL, tBE, tCE). It does not give a datasheet revision or clause numbers.

Reference Specification: Macronix MX25U6432FM2I02 QSPI NOR flash.

The QSPI Flash Controller is a generic phase sequencer. It does not decode a fixed opcode list. Firmware puts the opcode in command\_code and describes the phases in transfer\_config\_reg. The table lists the commands and behaviors the project document actually describes. A command that the document does not mention is not marked supported and is not marked unsupported. Clause numbers are not in the project document.

| <b>Feature (from reference spec)</b>  | <b>Spec Section/Clause</b>             | <b>Supported? (Yes / Partial / No)</b> | <b>Notes</b>                                                                                                 |
|---------------------------------------|----------------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Indirect command programming over APB | Not specified in project documentation | Yes                                    | Firmware writes the register bank and sets the start bit.                                                    |
| Write Enable (WREN)                   | Not specified in project documentation | Yes                                    | Named as a firmware step before program, erase, and status-write. Opcode not stated in the project document. |
| Page Program (PP, 0x02)               | Not specified in project documentation | Yes                                    | Indirect write. Data is preloaded through write_fifo_data. Up to 256 bytes in the described TX FIFO.         |
| Sector Erase (SE, 0x20)               | Not specified in project documentation | Yes                                    | Indirect command. Overlap with the cached XIP line invalidates that line.                                    |
| Block Erase (BE, 0xD8)                | Not specified in project documentation | Yes                                    | Indirect command. WIP remains high afterward; see the 50 ms / 300 ms comment in Section 6.7.                 |
| Write Status Register (WRSR, 0x01)    | Not specified in project documentation | Yes                                    | Used to set QE. tW = 40 ms. Opcode 0x01 is the value given in the initialization section.                    |
| Read Status Register (RDSR, 0x05)     | Not specified in project documentation | Yes                                    | Indirect command, and the XIP WIP poll.                                                                      |
| Read JEDEC ID (RDID, 0x9F)            | Not specified in project documentation | Yes                                    | Indirect read.                                                                                               |
| Fast Read (FAST_READ)                 | Not specified in project documentation | Yes                                    | Named indirect read. Opcode not stated in the project document. Dummy-cycle programming follows Section 10.  |
| Dual Output Read (DOR, 0x3B)          | Not specified in project documentation | Yes                                    | Indirect read. 2-wire data phase.                                                                            |
| Dual I/O Read (DIO, 0xBB)             | Not specified in project documentation | Yes                                    | Indirect read. 2-wire address and data.                                                                      |
| Quad Output Read (QOR, 0x6B)          | Not specified in project documentation | Yes                                    | Indirect read. 4-wire data phase. Requires QE.                                                               |

| Feature (from reference spec)        | Spec Section/Clause                    | Supported? (Yes / Partial / No) | Notes                                                                                                                          |
|--------------------------------------|----------------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Quad I/O Read (QIOR, 0xEB)           | Not specified in project documentation | Yes                             | Indirect read, and the XIP line fill. 4-wire address and data, with a mode phase and dummy cycles. Requires QE.                |
| Quad Input Page Program (QIPP, 0x32) | Not specified in project documentation | Yes                             | Indirect 4-wire page program. Requires QE.                                                                                     |
| XIP read over AHB-Lite               | Not specified in project documentation | Yes                             | 16-byte prefetch line. Miss: RDSR poll, then a 16-byte QIOR, with no firmware involvement.                                     |
| XIP write                            | Not specified in project documentation | No                              | Read-only. A write asserts hresp and does not modify the flash.                                                                |
| 1-wire, 2-wire, and 4-wire phases    | Not specified in project documentation | Yes                             | Selected per phase by transfer_config_reg. The value 11 is not defined.                                                        |
| 3-byte addressing                    | Not specified in project documentation | Yes                             | addr_byte_count = 0. address_bytes is 24 bits. The prefetch tag is addr[23:4].                                                 |
| 4-byte addressing                    | Not specified in project documentation | No                              | addr_byte_count = 1 is described as future scope. The current design supports 3-byte addresses because of the prefetch buffer. |
| Abort                                | Not specified in project documentation | No                              | control_signals MSB is named as abort and is future scope.                                                                     |
| Programmable SCLK divider            | Not specified in project documentation | No                              | No divider register. sclk frequency is the frequency driven onto the port.                                                     |
| Programmable CPOL/CPHA               | Not specified in project documentation | No                              | Fixed by construction. No mode-select register.                                                                                |

Other commands in the MX25U6432FM2I02 catalogue are *[Not specified in the existing project documentation]*. Because the engine is opcode-blind, an opcode the document does not discuss can still be placed in command\_code. That is not, by itself, a statement that the design supports that command.

## 9. Parameters

These are the parameters the project document states, with the values that document uses. No legal range is stated for any of them.

| Parameter | Legal Range | Default | Effect |
|-----------|-------------|---------|--------|
|-----------|-------------|---------|--------|



| Parameter             | Legal Range              | Default          | Effect                                                                                                                           |
|-----------------------|--------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------|
| LINE_SIZE             | Not explicitly specified | 16               | Bytes in the single prefetch line. The XIP path fetches and stores 16 bytes.                                                     |
| ADDR_WIDTH            | Not explicitly specified | 24               | Address width of the prefetch buffer. Matches the 3-byte flash address.                                                          |
| OFFSET_BITS           | Not explicitly specified | 4                | Low bits of the address inside the line. The tag is addr[23:4].                                                                  |
| MAX_WIP_POLLS         | Not explicitly specified | 16'hFFFF (65535) | Maximum RDSR polls in one XIP miss. Reaching it raises timeout_error.                                                            |
| TX FIFO ADDR_WIDTH    | Not explicitly specified | 8                | TX FIFO depth of 256 bytes, the largest page program described for the design.                                                   |
| RX FIFO ADDR_WIDTH    | Not explicitly specified | 5                | RX FIFO depth of 32 bytes. The read side drains during the transfer so a 256-byte read does not have to fit in the FIFO.         |
| async_fifo ADDR_WIDTH | Not explicitly specified | 5                | Default depth parameter of the shared FIFO primitive when an instance does not override it. The TX instance overrides it with 8. |

pclk = 50 MHz, sclk = 12.5 MHz, and the 4:1 ratio are the bring-up clocks in Section 4. They are not parameters and they are not programmed by a register.

## 10. Known Limitations

The following limitations and integration assumptions are stated in the project document.

- **Dummy-cycle count is programmed as one less than the value in the flash specification.** The design interprets the dummy-cycle field as one less than the datasheet count which is a design bug, but mending the changes will cause unpredictable error. Hence it is future scope
- Four-byte addressing is not implemented. addr\_byte\_count can represent a 4-byte address, but the current design supports 3-byte addresses because of the prefetch-buffer address width. Four-byte address is causes conflict with AHB Path .
- Abort is not implemented. The MSB of control\_signals is reserved for abort and is future scope.
- There is no clock-divider register. SCLK frequency is the frequency on the sclk port.
- CPOL and CPHA are fixed. There is no SPI-mode register.
- SCLK close to, or faster than, PCLK is not a supported clocking. The crossing structures are described as ratio-independent, but the latency margins, including the 4-cycle delayed done, were sized for a slower SCLK.
- pslverr is tied to 0. Undefined APB offsets 0xA–0xF do not return an error. A write to a busy-gated register is silently dropped and still completes with pready = 1 and pslverr = 0.

- `write_fifo_data` (offset 0x5) is not gated by `indirect_path_busy`. A write during a transaction is accepted. The TX FIFO is a preload-before-start buffer, not a mid-burst refill path.
- `hresp` for that rejection is a single-cycle placeholder. It is not the two-phase `HREADYOUT` error sequence required by the AHB-Lite specification. The project document identifies ARM\_IHI0033A as the specification to compare it with.
- `hburst` is latched and unused. Multi-line prefetch ahead is not implemented.
- The prefetch buffer is one 16-byte line. An access pattern that alternates between two lines refetches on every access.
- QE is not managed by the controller. Firmware must set QE once, with `WRSR`, and leave it set. XIP QIOR requires QE already set. Setting QE takes 40 ms (`tW`).
- The first command must wait `tVSL` = 800 microseconds after `presetn` is deasserted. The behavioral flash model refuses an earlier command.
- After a block erase, the document gives two different WIP times: around 50 ms in the XIP description, and `tBE` = 300 ms (with `tCE` up to about 18 s) in the limitations text. See the comment in Section 6.7. The XIP path polls WIP either way.
- `MAX_WIP_POLLS` (65535) has not been shown to cover a full block or chip erase. If an erase is in progress when an XIP fetch misses, the `RDSR` loop has to last for the whole erase without raising `timeout_error`. The poll period times 65535 has not been compared with `tCE`.
- APB wins every conflict, and there is no fairness mechanism. Back-to-back indirect transactions can hold off the XIP path for as long as they continue.